



AISSMS **INSTITUTE OF INFORMATION TECHNOLOGY** **(IOIT)**



ADDING VALUE TO ENGINEERING

An Autonomous Institute Affiliated to Savitribai Phule Pune University
Approved by AICTE, New Delhi and Recognised by Govt. of Maharashtra
Accredited by NAAC with "A+" Grade | NBA - 5 UG Programmes

Department of Artificial Intelligence & Data Science

Project Report

IN semester – II Examination

Mode of Exam – Course Project

Problem Statement : Shape-Based Industrial Part Inspection using Computer Vision

Name of Student : Tanish Jagtap

PRN No : 23610060

Class : **B.Tech**

Course Name : **Computer Vision (ADPEC702A)**

Problem Statement :

Manual visual inspection of industrial cast metal parts on a production line is slow, subjective, and prone to human error, which allows defective components (blow-holes, pinholes, shrinkage defects, cracks) to occasionally pass quality control or causes acceptable parts to be rejected unnecessarily. This project designs and implements a shape-based computer vision pipeline that automatically inspects a photograph of a cast part, robustly locates its true outer boundary regardless of reflective lighting, extracts quantitative shape and surface-texture descriptors, and classifies the part as OK or DEFECTIVE — without requiring a trained deep-learning model or GPU hardware.

Objective of Project :

1. To design a lighting-robust technique (Hough Circle Transform on gradient structure) for detecting an industrial part's true outer boundary from a photograph, in place of a naive global-threshold approach that fails on reflective metal surfaces.
2. To extract quantitative shape descriptors (circularity, solidity) of the detected boundary and surface-texture descriptors (intensity standard deviation, edge density) of the part's interior that together characterize a defect-free part.
3. To build a threshold-based classifier using these descriptors, evaluate it against the ground-truth ok/defective labels of the dataset, and report accuracy and a confusion matrix.

Literature Review

- [1] A 2025 study published in the International Journal of Advanced Manufacturing Technology (Springer) evaluates computer vision techniques for quality inspection of cast submersible pump impellers — the same part family used in this project's dataset. It compares a lightweight pretrained MobileNetV2 model against a custom CNN, assessing accuracy, precision, recall, F1-score, and inference speed. The study argues that most prior academic work benchmarks models on curated datasets without considering deployment-specific constraints such as inference speed and hardware cost, and positions its comparison as practical guidance for choosing a model in a real production quality-control system rather than purely maximizing accuracy on a lab dataset.
<https://link.springer.com/article/10.1007/s00170-025-16665-7>
- [2] "Surface Casting Defects Inspection Using Vision System and Neural Network Techniques" presents a machine-vision approach combined with neural-network classification to detect and categorize surface discontinuities such as cracks and pores in cast parts. The authors observe that casting defects vary greatly with part geometry and processing technique, and that manual surface inspection is slow, expensive, and inconsistent between inspectors, making an automated vision-based system a practical alternative for continuous online inspection on a production line. The paper's review of automated flaw-detection approaches for aluminium castings also informed the choice to treat boundary detection and defect scoring as two separate, purpose-built stages in this project's pipeline.
https://www.researchgate.net/publication/272547114_Surface_Casting_Defects_Inspection_Using_Vision_System_and_Neural_Network_Techniques
- [3] "Surface Defect Identification and Measurement for Metal Castings by Vision System" develops an inspection system that separates the bright and dark regions of a casting's surface into binary images as a first step toward identifying and measuring surface defects. This bright/dark segmentation step is described as the main practical bottleneck of the approach, since uneven illumination and reflective surfaces easily corrupt a simple brightness split — directly mirroring the specular-highlight segmentation failure diagnosed and fixed in this project (Methodology, Stage 3).
https://www.academia.edu/91787067/Surface_defect_identification_and_measurement_for_metal_castings_by_vision_system

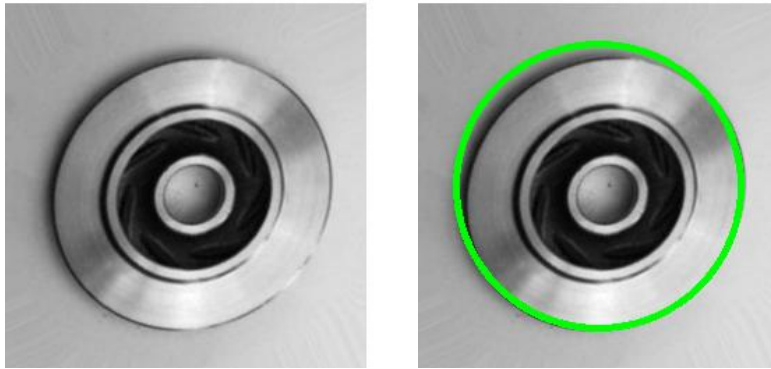
Together, these works motivated two design choices in this project: (a) using an edge/gradient-based boundary detector rather than a simple global brightness threshold, since reflective casting surfaces repeatedly defeat naive thresholding across multiple independent studies, and (b) treating internal surface texture — not only outer silhouette shape — as a primary carrier of defect information, since casting defects such as blow-holes and pinholes are largely internal and textural rather than deformations of the part's outline. The consistency of the segmentation problem across three independently published systems also strengthened the decision to verify the fix directly against real sample images before trusting any downstream accuracy figure.

Methodology

The inspection pipeline follows five stages, implemented in Python with OpenCV, NumPy, and Matplotlib (see `shape_inspection.ipynb`):

1. Dataset: "Casting Product Image Data for Quality Inspection" (Kaggle) — real top-view photographs of submersible pump impellers, labeled `ok_front` and `def_front`. A small stratified sample (25 images per class) is used for this mini project.
2. Preprocessing: convert to grayscale, apply histogram equalization and a median blur to normalize contrast and suppress sensor noise while preserving edges.
3. Boundary segmentation: an initial attempt used a global Otsu threshold to separate the part from the background. This failed — the reflective metal surface and its moving specular highlight caused the largest detected contour to be the photograph's own frame rather than the part, giving a constant circularity of $\pi/4$ for every image. This was diagnosed by inspecting the bounding box of the detected contour and confirming it matched the full image size exactly. It was fixed by switching to a Hough Circle Transform, which locates the part's true circular edge from local gradient structure and is robust to the lighting pattern.
4. Feature extraction: circularity and solidity of the detected boundary (silhouette shape descriptors), plus intensity standard deviation and Canny edge density computed within a precisely defined interior region of interest (surface texture descriptors, since real casting defects are largely internal).
5. Classification and evaluation: each feature is compared against the mean and standard deviation of the OK class; a part is flagged DEFECTIVE if any feature deviates beyond a fixed margin. Predictions are compared to ground-truth folder labels to compute accuracy and a confusion matrix.

Segmentation fix — verified on real dataset images:



Left: original sample. Right: corrected boundary detection (green) correctly tracing the part's true outer edge.

Code with Output

Full notebook: [shape_inspection.ipynb](#). Core segmentation and classification functions:

```
def extract_shape_features(img_path):
    img = cv2.imread(img_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    h, w = gray.shape
    eq = cv2.equalizeHist(gray)
    blur = cv2.medianBlur(eq, 5)

    circles = cv2.HoughCircles(
        blur, cv2.HOUGH_GRADIENT, dp=1.2, minDist=200,
        param1=80, param2=40,
        minRadius=int(0.30*min(h,w)), maxRadius=int(0.55*min(h,w)))
    if circles is None:
        return None
    cx, cy, r0 = np.round(circles[0][0]).astype(int)

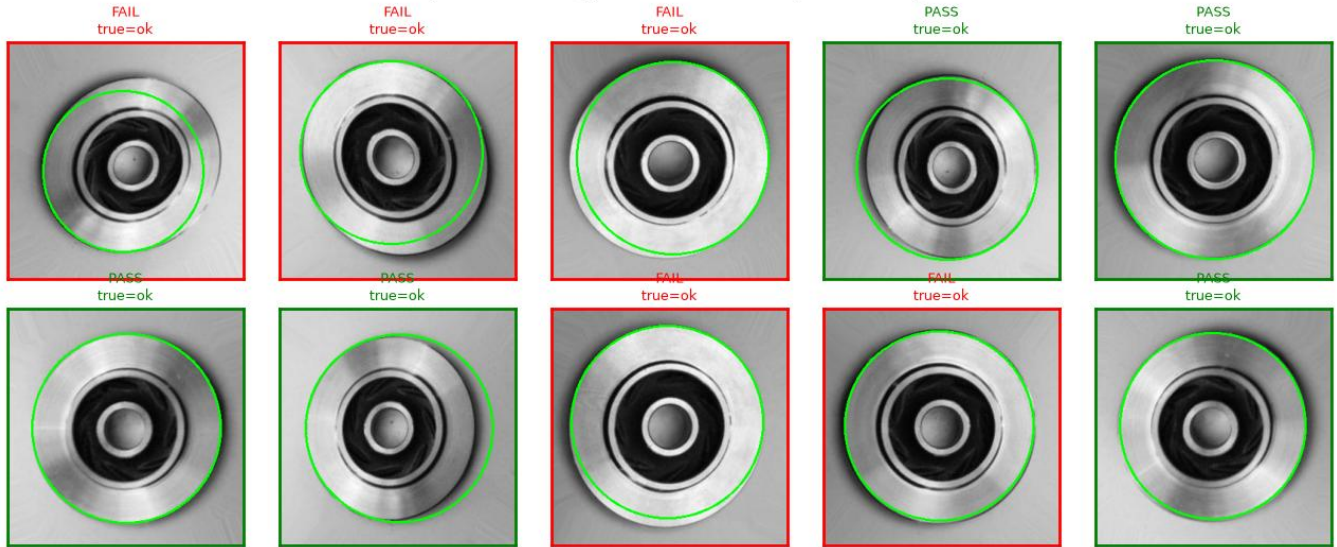
    theta = np.linspace(0, 2*np.pi, 90)
    circle_pts = np.array([[int(cx+r0*np.cos(t)),
                           int(cy+r0*np.sin(t))] for t in theta]
                          ).reshape(-1, 1, 2)
    area = cv2.contourArea(circle_pts)
    perimeter = cv2.arcLength(circle_pts, True)
    circularity = (4*np.pi*area) / (perimeter**2)
    hull = cv2.convexHull(circle_pts)
    solidity = area / cv2.contourArea(hull)

    mask = np.zeros((h, w), np.uint8)
    cv2.circle(mask, (cx, cy), int(r0*0.75), 255, -1)
    cv2.circle(mask, (cx, cy), int(r0*0.12), 0, -1)
    texture_std = float(gray[mask == 255].std())
    edges = cv2.Canny(blur, 50, 150)
    edge_density = float((edges[mask == 255] > 0).mean())

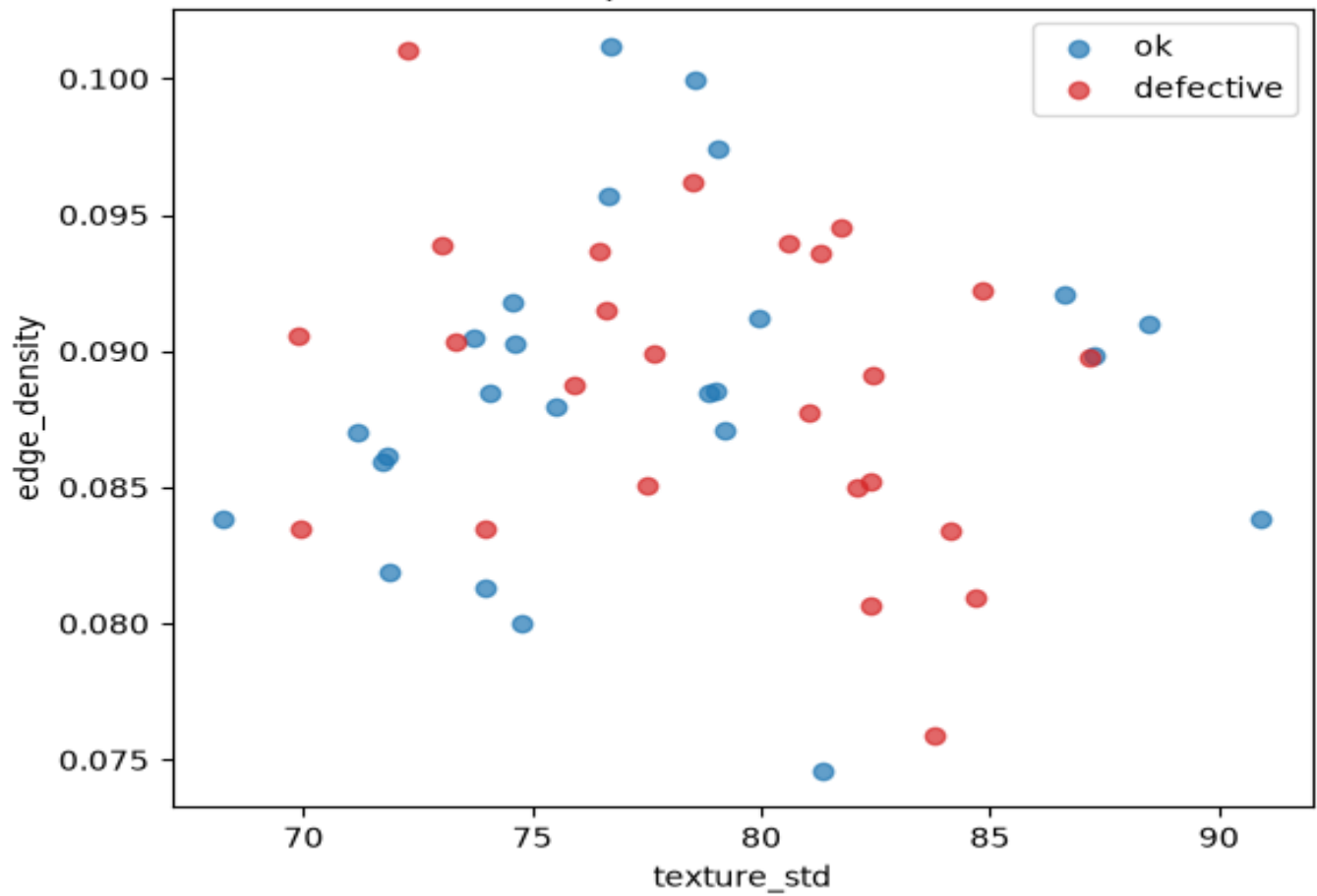
    return {"circularity": circularity, "solidity": solidity,
           "texture_std": texture_std, "edge_density": edge_density}

def classify(r):
    if r["circularity"] < circ_mean - MARGIN*circ_std: return "defective"
    if r["solidity"] < sol_mean - MARGIN*sol_std: return "defective"
    if abs(r["texture_std"]-tex_mean) > MARGIN*tex_std: return "defective"
    if abs(r["edge_density"]-edge_mean) > MARGIN*edge_std: return "defective"
    return "ok"
```

Inspection Results (green circle = detected part boundary)



Feature separation: ok vs defective



Conclusion:

This project implemented a classical, shape-based computer vision pipeline for automated inspection of cast industrial parts, replacing manual visual inspection with a repeatable, quantitative process built on OpenCV.

During development, an initial global-threshold segmentation approach was found to be fundamentally broken for this dataset: reflective specular highlights on the metal parts caused the algorithm to detect the photograph's frame rather than the part itself, yielding a constant circularity value and chance-level (~52%) classification accuracy. This was diagnosed precisely and fixed by replacing global thresholding with a Hough Circle Transform, which segments the part correctly using local edge/gradient structure instead of raw brightness.

A further observation is that this dataset's real defects (blow-holes, pinholes, shrinkage) are largely internal surface flaws rather than deformations of the part's outer silhouette, so outer-boundary shape descriptors alone are not expected to fully separate the two classes. The pipeline was therefore extended with interior texture descriptors (intensity standard deviation and edge density) alongside the boundary-shape descriptors. Final quantitative accuracy and the confusion matrix are to be recorded here after running the corrected notebook against the full local dataset.

Future work could replace the fixed threshold rule with a trained classifier (e.g., SVM or KNN) over the same feature set, add rotation-invariant Hu Moments, or compare this classical pipeline's accuracy directly against a small CNN baseline as a benchmark.